

Chapter 6. 사람-에이전트 반복 개정 루프

이 챕터에서 배우는 것 (이런 분이 먼저 읽으면 좋다: "사람이 피드백을 주면 AI가 반영한다"는 흔한 기능처럼 들리지만, 그 반복을 안전하게 설계하는 게 왜 까다로운지 궁금한 분)

- `run_review_loop()` 가 저자 피드백을 에이전트에게 어떻게 넘기는지
- "무한 반복"을 막는 두 겹의 안전장치가 왜 하나로써는 부족한지
- 이 루프가 왜 `conversation_eval` (다회 대화 전용 데코레이터)이 아니라 매 라운드 `@agent_eval` 을 개별 호출하는 방식을 택했는지

6.1 협업의 세 번째 형태 — 사람이 루프의 일부다

지금까지 다룬 두 협업(4장의 순차 파이프라인, 5장의 감독자-작업자)은 전부 에이전트 끼리의 협업이었다. `review_loop.py` 는 다르다. **저자(사람)가 루프의 한 참여자**다. 기획안이나 목차 초안을 보여주고, 저자가 Enter(승인)를 누르거나 수정 요청을 입력하면, `revise()` 가 그 피드백을 반영해 다시 쓴다. 승인할 때까지 이 왕복이 계속된다.

```
def run_review_loop(
    *, kind: str, initial_md: str, revise_fn: ReviseFn,
    render: Callable[[str], None], ask_feedback: Callable[[], str],
    on_feedback: Callable[[str], None] | None = None,
) -> str:
    current = initial_md
    round_no = 0
    while True:
        render(current)
        if round_no >= MAX_REVIEW_ROUNDS:
            return current
        feedback = ask_feedback()
        if not feedback or feedback.strip().lower() in {"y", "yes", "승인", "ok"}:
            return current
        if on_feedback is not None:
```

```

on_feedback(feedback)
round_no += 1
current = revise_fn(current_md=current, feedback=feedback, round_no=round_no, kind=kind, ground_truth=feedback)

```

6.2 두 겹의 안전장치 — 왜 하나로 부족인가

이 루프는 무한히 돌 수 있는 구조다. 저자가 계속 수정을 요청하면 계속 다시 쓴다. Book-forge는 이 위험을 **두 층**으로 막는다.

층	메커니즘	막는 것
애플리케이션 레벨	<code>MAX_REVIEW_ROUNDS = 5</code>	라운드 수 자체가 5를 넘으면 최신 초안으로 강제 진행
Harness 레벨	<code>LoopDetectionConfig(consecutive_repeat_threshold=3)</code>	같은 피드백이 반복 되는 것(저자가 실수로 같은 요청을 계속 입력하는 경우)

이 둘은 서로 다른 문제를 막는다는 것이 핵심이다. `MAX_REVIEW_ROUNDS`는 "얼마나 여러 번 도는가"를, `LoopDetectionConfig`는 "같은 내용이 반복되는가"를 본다. 저자가 매번 다른 유효한 피드백을 5번 넘게 준다면 `MAX_REVIEW_ROUNDS`가 막는다. 저자가 같은 피드백을 3번 연속 입력한다면(예: 이전 개정이 반영이 안 됐다고 착각해 같은 말을 반복) `LoopDetectionConfig`가 먼저 잡는다. 코드 주석은 이 관계를 명시적으로 설명한다.

"`LoopDetectionConfig`는 '같은 피드백의 반복'만 잡지, 라운드 수 자체를 제한하지 않으므로 이 상한은 애플리케이션 레벨의 별도 안전장치다."

6.3 왜 `conversation_eval`이 아니라 라운드마다 `@agent_eval`인가

`review_loop.py`의 파일 최상단 docstring에 이례적으로 자세한 설명이 있다. `conversation_eval`(Agent-Evaluator SDK가 제공하는 다회 대화 전용 데코레이터)이 이 상황에 더 자연스러워 보일 수 있지만, 실제 SDK 소스(`decorators.py`)를 확인한 결과 **31개 Harness Config 전부가 시그니처로만 받아질 뿐 평가에 실제로 반영**

되지 않는다는 것을 발견했다(`_CONVERSATION_EVAL_UNUSED_HARNESS_PARAMS` 튜플을 직접 세면 31개). `LoopDetectionConfig` 가 실제로 작동하려면 각 라운드가 독립된 `TaskResult` 로 기록되어야 하는데, `conversation_eval` 은 그 구조를 만들지 않는다.

```
@agent_eval(
    monitor, task_type="planning", question_arg="feedback",
    task_id_fn=lambda args, kwargs: f"review_{kwargs['kind']}_{kwargs['round_no']}",
    loop_detection=LoopDetectionConfig(consecutive_repeat_threshold=3),
)
def revise(current_md: str, feedback: str, round_no: int, kind: str, ground_truth: str = "") -> tuple[str, EvalMetadata]:
    ...
```

`task_id_fn` 이 `f"review_{kind}_{round_no}"` 처럼 라운드 번호를 태스크 ID에 새겨 넣는다. 이래야 `LoopDetectionConfig` 가 "같은 종류(kind)의 몇 번째 라운드에서 반복이 일어났는가"를 태스크 단위로 정확히 판정할 수 있다. 이 설계 결정은 이 책 전체에서 반복될 원칙을 보여준다. **"더 편해 보이는 API"가 항상 "실제로 원하는 신호를 주는 API"는 아니다.** 소스 코드를 직접 확인하지 않고 문서상 그럴듯한 API를 골랐다면, `LoopDetectionConfig` 를 설정해도 조용히 무시했을 것이다.

 **QA 관리자 TIP:** `task_id_fn` 은 `(args, kwargs)` 를 받으므로, 호출부는 반드시 키워드 인자로 `revise_fn` 을 호출해야 한다(`revise(current_md=..., feedback=..., round_no=..., kind=...)`). 위치 인자로 호출하면 `task_id_fn` 이 `kwargs['kind']` 를 찾지 못해 예외가 난다. 이런 종류의 "데코레이터 설정과 호출 방식 사이의 암묵적 계약"은 코드 리뷰에서 놓치기 쉬운 지점이다.

6.4 피드백이 그대로 채점 근거가 된다

`revise()` 호출에서 `ground_truth=feedback` 이 눈에 띈다. 개정된 결과가 "저자가 요청한 것을 실제로 반영했는가"를 채점할 근거로, 저자의 피드백 문장 자체를 쓴다. 이는 6장이 다루는 협업의 본질을 정확히 보여준다. **사람의 발화가 곧 다음 채점의 정답 기준이 된다.** 5장의 리뷰어들이 자기 역할(`AgentRoleConfig`)로 평가받았다면, 이 루프의 `revise()` 는 "사람이 원한 것을 실제로 했는가"로 평가받는다.

직접 해보기

`book-forge plan <slug> --revise` 로 승인 루프를 직접 돌려보되, 일부러 같은 피드백을 3번 연속 입력해보자.

`LoopDetectionConfig(consecutive_repeat_threshold=3)` 가 실제로 개입해 차단하는 것을 직접 관찰할 수 있다 (§ 6.2). **여러분이 사람 피드백을 반영해 재생성하는 루프를 만든다면**: "라운드 수 상한"과 "같은 내용 반복 탐지"라는 두 안전장치를 반드시 함께 두라. 하나만으로는 서로 다른 위험 (§ 6.2의 표)을 놓친다.

이 챕터의 핵심

- **사람이 루프의 참여자로 명시적으로 설계돼 있다.** `ask_feedback()` / `render()` 콜백이 그 접점이다.
- **무한 반복 방어는 두 층으로 나뉜다.** 라운드 수 상한(애플리케이션)과 반복 피드백 탐지(Harness)는 서로 다른 문제를 막는다.
- **SDK API를 문서만 보고 고르면 안 된다.** `conversation_eval` 대신 라운드별 `@agent_eval` 을 택한 것은 실제 SDK 소스를 확인해 내린 결정이다.

참고 자료

- `src/book_forge/agents/review_loop.py` — 전체
- `src/book_forge/cli/commands/new_cmd.py` · `plan_cmd.py` — `run_review_loop()` 실제 호출 지점

다음 챕터는 마지막 협업 패턴 — 파일로 저장되는 지식창고를 매개로, 집필 세션이 끝난 뒤에도 이어지는 간접 협업을 다룬다.